



MODULE :

Le Cloud Computing & DevOps

Pr. F. Benabbou
Master DSBD
Faculté des Sciences Ben M'Sik Casablanca





TABLE OF CONTENTS

01 CLOUD COMPUTING

- Introduction générale
- La Virtualisation
- Les concepts de base du Cloud Computing
- Technologies émergentes du CC : Edge, Fog, ...
- Étude de cas et projet pratique

02 DevOps & Cloud

- La philosophie DeVops
- Version control systems (git)
- Intégration Continue CI
- Déploiement Continu CD
- Tests automatisés
- Infrastructure en tant que Code (IaC)
- Étude de cas et projet pratique



02

Gérer les déploiements de l'infrastructure avec Ansible et Terraform

Infrastructure en tant que Code (IaC)



- Dans les approches classiques, si on veut installer le paquet nginx sur un ensemble de serveurs distants, l'admin doit répéter l'opération sur les différents serveurs
- L'administrateur va effectuer les étapes suivantes:
 - Connexion à chacun des serveurs.
 - Configuration de chaque serveur.
 - Installation de nginx sur chacun.
- Cette approche a des inconvénients:
 - Installation individuelle, Perte de temps
 - Erreurs humaines
 - Difficulté de reproduction

```
$ ssh www1.example.com
www1$ sudo vi /etc/resolv.conf
www1$ sudo apt-get install nginx
:
$
$ ssh www2.example.com
www2$ sudo vi /etc/resolv.conf
www2$ sudo apt-get install nginx
:
$
$ ssh www3.example.com
www3$ sudo vi /etc/resolv.conf
www3$ sudo apt-get install nginx
:
:
: etc ...
```

Automatiser avec script shell

- On peut écrire un script shell ou les tâches seront dans une boucle.
- Mais cela reste difficile à écrire et difficile à maintenir quand on veut personnaliser la configuration sur chaque serveur.
- Il y a un risque de provoquer des erreurs

```
#!/bin/sh

HOSTS="
www1.example.com
www2.example.com
www3.example.com
"

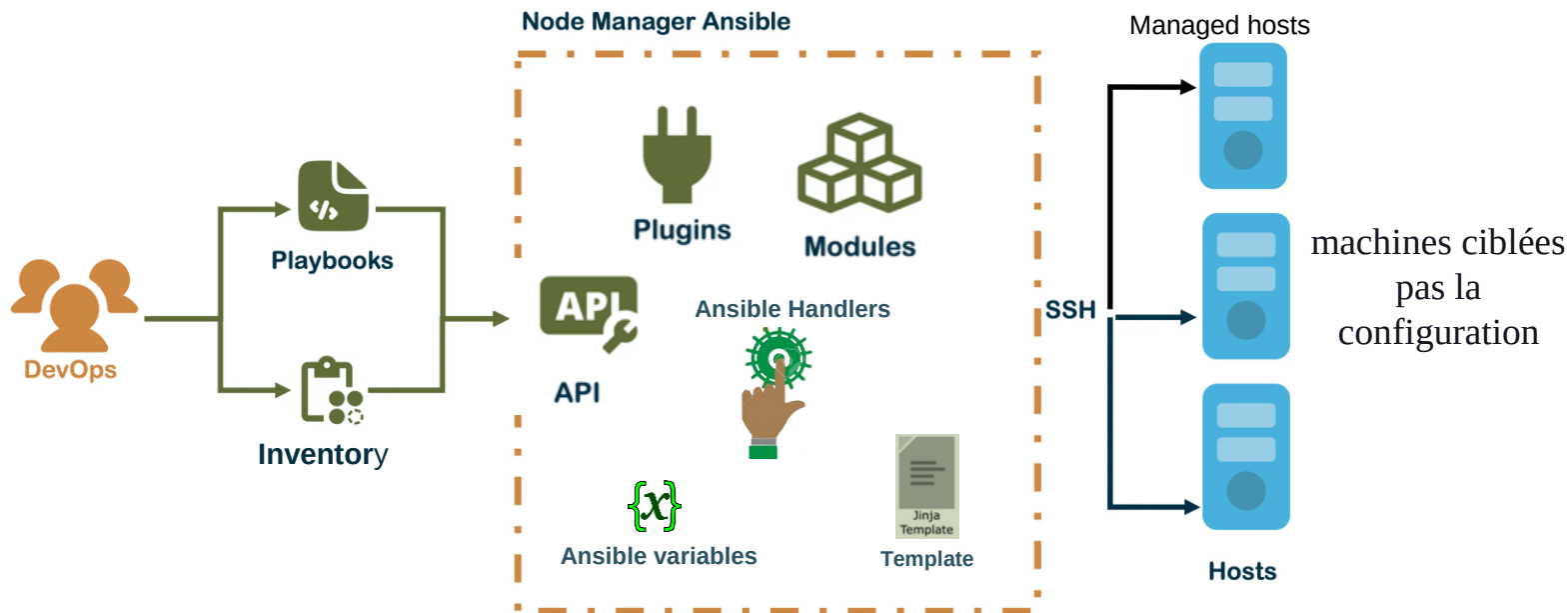
for host in $HOSTS
do
    # Copy DNS settings to all servers
    scp resolv.conf $host:/etc/resolv.conf

    # Install Nginx
    ssh $host "sudo apt-get install nginx"
done
```

- Ansible est principalement un outil de gestion de **configuration**, mais il gère l'Infrastructure (IaC).
- Ansible permet d'automatiser différentes tâches
 - Le **Provisioning** (IaC) : il interagit avec les APIs de ces fournisseurs Cloud, comme AWS ou Azure pour créer des ressources comme des serveurs, du stockage ou des réseaux, etc.
 - La **Gestion de Configuration** : Installer des services (Nginx, MySQL), gérer les utilisateurs et les fichiers de configuration sur ces ressources.
 - L'**Orchestration** : Coordonner des tâches sur plusieurs serveurs (mettre à jour la base de données, stockage, des réseaux, des pare-feu).
- Ansible est une technologie sans agent (agentless), ce qui signifie qu'elle n'installe aucun logiciel sur les nœuds qu'elle gère.

Architecture Ansible

- On utilise des fichiers yaml nommé **playbook**, pour décrire l'état souhaité de l'infrastructure, et Ansible s'occupe de l'exécuter automatiquement.



- Appelé aussi, control node, est le poste où est installé Ansible, depuis laquelle on lance les commandes Ansible.
- Les responsabilités du Control Node
 - ✓ Interprétation : Il lit les fichiers YAML (Playbooks) et les transforme en instructions exécutables.
 - ✓ Orchestration : Il décide de l'ordre d'exécution (ex: mettre à jour le serveur A, puis le serveur B).
 - ✓ Génération de modules : Il prépare les petits scripts Python nécessaires à chaque tâche.
 - ✓ Centralisation des logs : Il récupère les résultats de chaque machine et affiche un résumé global à l'administrateur.

Composants clés d'Ansible

- L'architecture d'Ansible repose sur un ensemble d'éléments clé:
 - ✓ Inventory : liste des machines
 - ✓ Playbook : fichier principal d'automatisation
 - ✓ Gather Facts: collecte automatiquement des informations sur les machines.
 - ✓ Play : association entre machines et tâches
 - ✓ Task : action à exécuter
 - ✓ Module : outil utilisé par la tâche
 - ✓ Variables : paramètres dynamiques
 - ✓ Template : génération de fichiers dynamiques
 - ✓ Handler : action déclenchée après modification
 - ✓ Roles: c'est une structure organisée et réutilisable

- Le fichier inventaire `inventory.yml` contient une liste des hôtes à gérer, avec leurs adresses IP, noms d'hôte, groupes, etc.
- On distingue deux types d'inventaire :
 - ✓ l'inventaire statique constitué de fichier(s) plats.
 - ✓ l'inventaire dynamique fourni par un système centralisé. Exemple L'inventaire AWS EC2.

```
all:                                #définit le groupe global
  hosts:                            #liste les hôtes
    server1:
      ansible_host: 192.168.1.10
    server2:
      ansible_host: 192.168.1.11
  children:                         #création du groupe logique
    web_servers:
      hosts:                        #Hôtes du groupe web_servers
        server1:
        server2:
```

Playbooks

- Le Playbook il contient une suite ordonnée de tâches destinées à configurer ou déployer un parc de machines.
- Le playbook décrit la situation souhaitée de l'infrastructure, et Ansible s'occupe de mettre en œuvre les changements nécessaires pour atteindre cet état.
- Exemple : installer le paquet apache sur tous les hôtes appartenant au groupe web_servers.

```
- name: Installer Apache
hosts: web_servers
tasks:
  - name: Installer le paquet Apache
    ansible.builtin.apt:
      name: Apache
      state: present
  - name: Démarrer le service Apache
    service:
      name: Apache
      state: started
```

- name définit la tâche principale.
hosts: web_servers : indique le groupe d'hôtes concerné par cette tâche, web_servers défini dans l'inventaire Ansible.
tasks: définit les tâches qu'Ansible doit exécuter sur les hôtes ciblés, ici on a 2 tâches : installer et démarrer le serveur.

- C'est l'unité de base de l'automatisation pour exécuter une tâche précise
- Son rôle peut être : installer des paquets, configurer des services, gérer les utilisateurs et fichiers, etc.
- Il existe plus de 1000 modules, voici des exemples :
 - ✓ Système : `apt` (`ansible.builtin.apt`), `yum`, `service`, `user`
 - ✓ Réseau : `ping`, `setup`
 - ✓ Fichiers : `copy`, `file`, `template`
 - ✓ Conteneurs : `Docker`, `Kubernetes`
 - ✓ Cloud : `ec2`, `azure_rm`, `gcp`

Exemple de playbook

- Ce playbook utilise plusieurs modules de Ansible pour installer et configurer Apache HTTP Server sur les serveurs du groupe web-servers.

```
- name: Déployer Apache sur les
  serveurs web
hosts: web-servers
become: yes      #Exécute les tâches
                  #avec privilèges sudo
tasks:
- name: MAJ le cache des paquets
  apt:
    update_cache: yes
- name: Installer Apache
  apt:
    name: apache2
    state: present
```

```
- name: Démarrer et activer Apache
  service:
    name: apache2
    state: started
    enabled: yes    # Active le service
- name: Déployer une page web simple
  copy:              # Créer une page HTML
    content: "<h1>Bienvenue sur mon
serveur Apache !</h1>"
    dest: /var/www/html/index.html
- name: Ouvrir le port 80
  ufw:                # gère le firewall UFW
    rule: allow
    port: "80"
    proto: tcp
```

Les faits d'Ansible



ANSIBLE

- `gather_facts` est une option qui permet à Ansible de collecter automatiquement des informations sur les machines cibles avant d'exécuter les tâches.
 - ✓ Système d'exploitation (Ubuntu, CentOS...)
 - ✓ Adresse IP
 - ✓ Nom de la machine
 - ✓ CPU / RAM
 - ✓ Disques
 - ✓ Interfaces réseau
- Ces informations sont stockées dans des variables qu'on nomme **facts**, et peuvent être utilisées lors de l'exécution du playbook pour personnaliser les configurations.

```
- name: Installer un serveur web selon l'OS
hosts: web-servers
become: yes
gather_facts: yes           # pour détecter l'OS

tasks:
  - name: Installer Apache sur Debian
    apt:
      name: apache2
      state: present         #vérifier paquet installé
      update_cache: yes     # apt-get update
      when: ansible_os_family == "Debian"
  - name: Installer Apache sur RedHat
    yum:
      name: httpd
      state: present
      when: ansible_os_family == "RedHat"
```

- Les variables permettent de stocker des informations et de rendre les playbooks flexibles pour différents environnements.
 - ✓ Variables **systèmes** (facts)
 - Récupérées automatiquement avec `gather_facts: yes`
 - Exemples : `ansible_hostname`, `ansible_os_family`, `ansible_default_ipv4.address`
 - ✓ Variables **utilisateurs**
 - Définies par l'utilisateur pour personnaliser le playbook
 - Deux façons de les définir :
 - Directement dans le playbook avec `vars`
 - Dans un fichier externe (ex: ``vars/main.yaml``) et inclus dans le playbook

■ Exemples 1: Reprenons l'exemple du déploiement d'un serveur Apache.

```
- name: Installer Apache de manière optimisé
hosts: web-servers
become: yes
gather_facts: yes
vars:
  apache_package: "{{ 'apache2' if ansible_os_family == 'Debian' else 'httpd' }}"
  apache_service: "{{ apache_package }}"

tasks:
  - name: Installer Apache
    package:
      name: "{{ apache_package }}"
      state: present

  - name: Démarrer Apache
    service:
      name: "{{ apache_service }}"
      state: started
      enabled: yes
```

- Exemples 2: playbook utilisant des variables (vars) pour créer plusieurs fichiers personnalisés sur plusieurs serveurs

```
- name: Créer des fichiers texte personnalisés sur plusieurs serveurs
hosts: all
become: true
gather_facts: yes
vars:                                # Définition des variables utilisateur
    fichier_path: /tmp/fichier_test.txt
    fichier_content: |                #indiquer que ce qui suit est un bloc texte multi-ligne
        Bienvenue sur {{ ansible_hostname }}!
        Ce fichier a été créé automatiquement par Ansible.
tasks:
    - name: Créer les fichiers définis dans les variables
      copy:
        dest: "{{fichier_path }}"      # utilisation de la variable fichier_path
        content: "{{fichier_content }}" # utilisation de la variable fichier_content
```

Variables Système Ansible

Variable	Description
<code>ansible_hostname</code>	Nom de l'hôte
<code>ansible_distribution</code>	Distribution Linux (ex. Ubuntu, CentOS)
<code>ansible_distribution_version</code>	Version de la distribution Linux
<code>ansible_os_family</code>	Famille du système d'exploitation (Debian, RedHat, etc.)
<code>ansible_default_ipv4.address</code>	Adresse IP principale
<code>ansible_processor</code>	Liste des processeurs
<code>ansible_memtotal_mb</code>	Mémoire totale en Mo
<code>ansible_architecture</code>	Architecture du système (x86_64, etc.)

- Un template est un fichier texte (souvent .j2 pour Jinja2)
- **Jinja2** est un moteur de templates utilisé dans Ansible pour créer des fichiers de configuration dynamiques et réutilisables pour des applications telles que Nginx, Apache et les bases de données en utilisant des variables.
- Un template permet de générer automatiquement des fichiers de configuration adaptés à chaque serveur ou environnement.
- Exemple :

■ La template Motdemarre.j2

```
Bienvenue sur le serveur {{ ansible_hostname }} !  
Utilisateur : {{ ansible_user }}  
Système : {{ ansible_os_family }}  
Date : {{ ansible_date_time.date }}
```

Bonne journée !

■ Utiliser la template dans le playbook

```
- hosts: serveurs  
  gather_facts: yes  
  
  tasks:  
    - name: Déployer le message du jour via template  
      template:  
        src: Motdemarre.j2  
        dest: /tmp/motd      # fichier généré sur le serveur: Message of the day
```

- Problématique : Imaginez que vous devez installer un **serveur Web** sur **10** projets différents, vous devez écrire le même code dans 10 Playbooks différents:
 - ✓ Constitue une perte du temps,
 - ✓ Possible de faire des erreurs de copier-coller.
 - ✓ Peut être compliqué si vous devez modifier les 10 fichiers le jour où la configuration change.
- Un role est une structure standardisée pour organiser les tâches, fichiers, templates, variables, handlers, etc., autour d'une fonctionnalité précise de manière à pouvoir le réutiliser facilement sur différents projets ou serveurs.
- Un role est associé à une **seul activité** par exemple : installation d'un serveur Apache ou configurer une BD.

Roles : comment déployer Apache

- Exemple : comment créer un rôle Ansible pour déployer Apache de manière claire et réutilisable

- ✓ En ligne de commande tape pour créer un rôle « apache »:

```
ansible-galaxy init apache
```

- ✓ Cette commande va créer automatiquement la structure standardisée d'un rôle :

```
apache/
├── tasks/
│   └── main.yml          # Tâches principales du rôle
├── templates/           # Fichiers template (.j2)
├── files/                # Fichiers statiques à copier
├── vars/
│   └── main.yml          # Variables spécifiques au rôle
├── defaults/
│   └── main.yml          # Valeurs par défaut
├── handlers/
│   └── main.yml          # Actions à déclencher (ex: restart service)
└── meta/
    └── main.yml          # Informations sur le rôle (auteur, dépendances, etc.)
```

Roles: comment déployer Apache

- créer un rôle Ansible pour déployer Apache

- ✓ defaults/main.yml

```
apache_port: 80
```

- ✓ vars/main.yml

```
document_root: /var/www/html  
index_file: index.html
```

- ✓ tasks/main.yml

Roles: comment déployer Apache

- Exemple : créer un rôle Ansible pour déployer Apache de manière claire et réutilisable

✓ tasks/main.yml

```
- name: Déterminer le nom du paquet Apache, et document root selon l'OS
  set_fact:
    apache_service_name: "{{ 'apache2' if ansible_os_family == 'Debian' else 'httpd' }}"
    apache_package: "{{ 'apache2' if ansible_os_family == 'Debian' else 'httpd' }}"
    document_root: "{{ '/srv/www/htdocs' if ansible_os_family == 'OpenSUSE' else '/var/www/html' }}"
- name: Installer Apache
  package:
    name: "{{ apache_package }}"
    state: present
- name: Démarrer Apache
  service:
    name: "{{ apache_service }}"
    state: started
    enabled: yes
- name: Déployer la page index via la template index.html.j2
  template:
    src: index.html.j2
    dest: "{{ document_root }}/{{ index_file }}"
  notify: redémarrer apache    #le handler se déclenche que s'il y a changement du fichier
```

Roles: comment déployer Apache

- Exemple : créer un rôle Ansible pour déployer Apache de manière claire et réutilisable

✓ handlers/main.yml

```
- name: "redémarrer apache"           #Contient les actions déclenchées par notify
- service:
    name: "{{ apache_service_name }}"  # variable définie dans tasks ou defaults
    state: restarted
```

✓ Templates/index.html.j2

```
<!DOCTYPE html><html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Bienvenue sur {{ ansible_hostname }}</title>
  <style>
    body {font-family: Courier New, monospace;background-color: #ffffff;color:#000000;
    }
    h1 {color: #2a7ae2;}
    p {font-size: 14px;}
  </style>
</head>
<body>
  <h1>Bienvenue sur {{ ansible_hostname }}</h1>
  <p>Site déployé automatiquement par Ansible.</p>
  <p>Port du serveur Apache : {{ apache_port }}</p>
  <p>Document root : {{ document_root }}</p>
</body>
</html>
```

Roles: comment déployer Apache

- Exemple : créer un rôle Ansible pour déployer Apache
 - ✓ playbook complet (deploy_apache.yaml) qui utilise le rôle apache que nous avons défini

```
- name: Déploiement Apache sur tous les serveurs web
  hosts: webservers
  become: yes           # pour exécuter les tâches avec privilèges root
  gather_facts: yes

roles:
  - apache
```

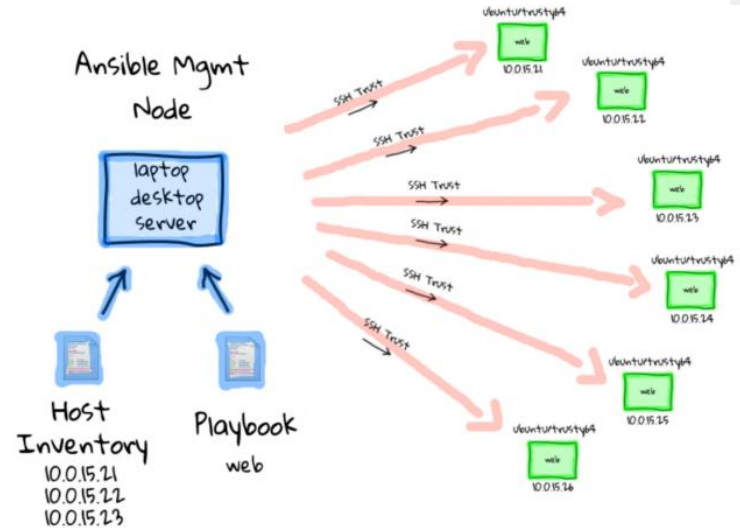
- ✓ Ansible va chercher le rôle apache dans le dossier roles/ et exécuter automatiquement
 - tasks/main.yam
 - handlers/main.yaml si nécessaire
 - templates/index.html.j2 pour générer la page d'accueil

Fonctionnement d'Ansible



ANSIBLE

- Ce diagramme illustre le flux d'exécution d'un playbook Ansible :
 - Le node manager Ansible récupère le playbook et l'inventaire.
 - L'inventaire est chargé pour identifier les hôtes à cibler.
 - Pour chaque hôte, il exécute les tâches définies dans le playbook.
 - Les résultats de l'exécution sont retournés au Node manager.



Ansible & cloud



- Ansible peut être utilisé pour **gérer l'infrastructure** en cloud.
- Il propose plusieurs **modules** spécialement conçus pour travailler avec différents fournisseurs de cloud.
- Ces modules offrent diverses fonctionnalités, telles que la création d'instances, la fermeture d'instances, l'attachement de volumes de stockage, la gestion de groupes de sécurité, etc.
- Parmi les modules cloud les plus populaires, on peut citer :
 - AWS: `ec2`, `ec2_instance`, `ec2_vpc`, `ec2_security_group`.
 - Google Cloud (GCP): `gcp_compute_instance`, `gcp_storage_bucket`.
 - Azure : `azure_rm_virtualmachine`, `azure_rm_resourcegroup`, `azure_rm_networkinterface`.
 - openstack: `os_server`, `os_network`, `os_image`.
 - DigitalOcean: `digital_ocean_droplet`, `digital_ocean_volume`.

Les avantages d'Ansible



SIMPLE

Ansible utilise un langage de description de configuration (YAML) très lisible, ce qui le rend accessible même aux débutants.



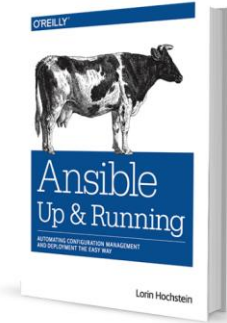
COMMUNAUTE

Il bénéficie d'une communauté active et de nombreux modules créés par la communauté. Top 10 open source projects in 2017,



AGENTLESS

Il ne nécessite pas d'agent logiciel à installer sur les machines à gérer, ce qui simplifie la complexité de déploiement.



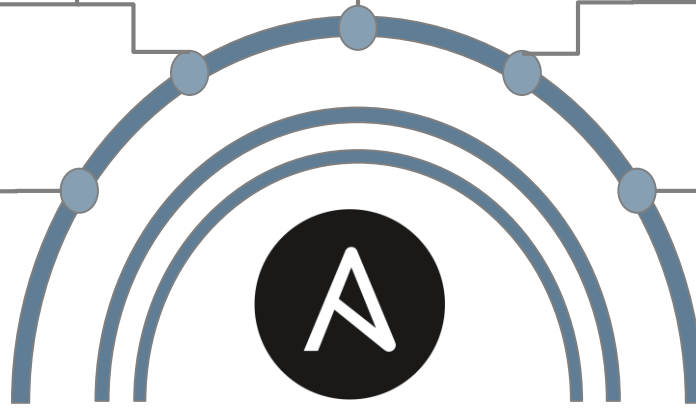
POWERFUL

Il offre une large gamme de **modules** pour automatiser une multitude de tâches, de l'installation de paquets à la configuration de services complexes.



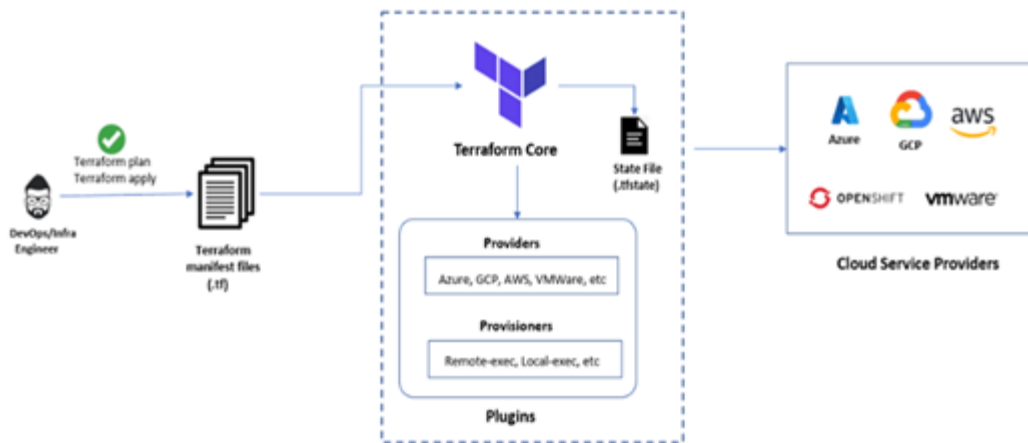
Idempotents

Les playbooks Ansible sont idempotents, ce qui signifie qu'ils peuvent être exécutés plusieurs fois sans modifier l'état du système s'il est déjà dans l'état souhaité.



Terraform

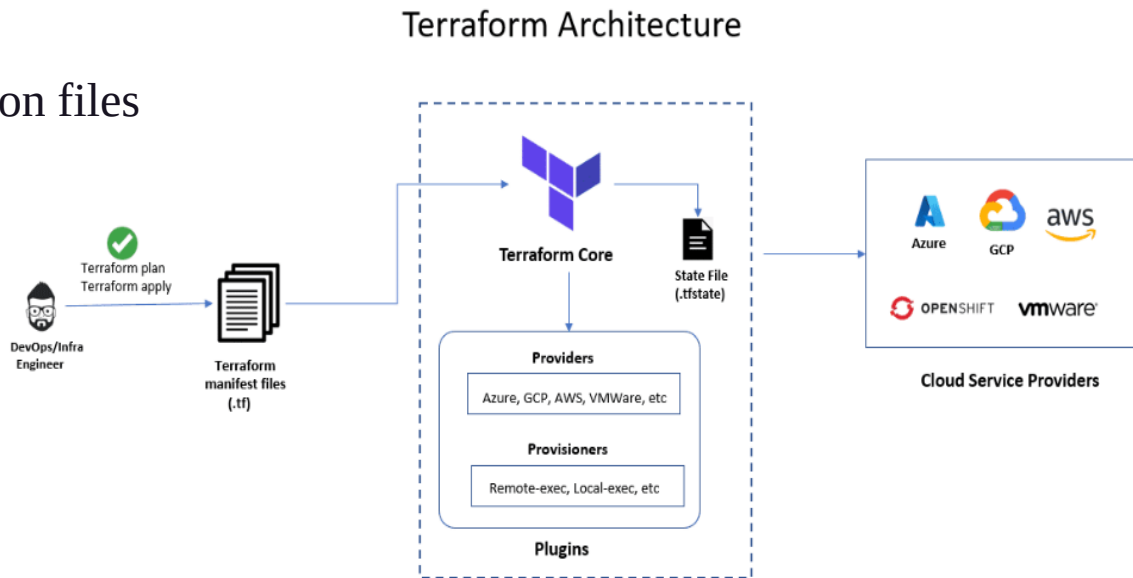
- Terraform est un outil Infrastructure as code (IaC), open source, développé en 2014 par HashiCorp.
- Il permet de créer, modifier, supprimer et gérer des infrastructures (serveurs, des réseaux et des bases de données) cloud ou sur site en utilisant des fichiers de configuration écrit avec un langage déclaratif (HCL, Hashicorp Configuration Language).
- L'architecture de Terraform se compose principalement des éléments suivants :
 - Terraform Core
 - Provider plugins
 - State file
 - Configuration files





Composant de Terraform

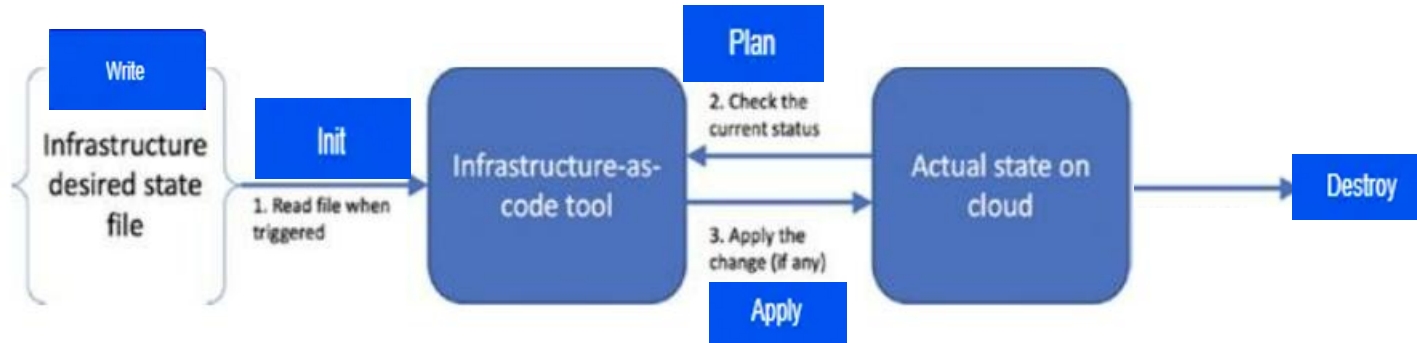
- L'architecture de Terraform se compose principalement des éléments suivants :
 - Terraform Core
 - Provider plugins
 - State file
 - Configuration files



Terraform Core



- Le Terraform Core est l'élément principale de Terraform qui orchestre l'ensemble des opérations : la configuration, les providers et l'infrastructure.
- Il gère les interactions entre la configuration, les plugins (providers), et l'état.
- Workflow Terraform
 - Write: écrire la configuration.
 - Init: initialise la configuration pour installer les dépendances nécessaires.
 - Plan: scan l'infrastructure existante et la compare avec l'état désirée
 - Apply: applique les changements à l'infrastructure réelle.
 - Destroy: libère toute l'infrastructure





- Un provider Terraform est un plugin qui permet à Terraform de communiquer avec une plateforme externe (cloud, SaaS ou infrastructure locale).
- Son rôle est de :
 - ✓ Connecter Terraform aux API des plateformes
 - ✓ Permettre de créer, lire, modifier et supprimer des ressources
 - ✓ Fournir les types de ressources spécifiques à chaque plateforme
- Exemples de providers courants :
 - ✓ AWS : Gère les ressources comme les instances EC2, S3, etc.
 - ✓ Google Cloud : Gère les ressources GCP comme Compute Engine et Cloud Storage.
 - ✓ AzureRM : Gère les ressources Azure.
 - ✓ Outscale : Gère les ressources dans le cloud Outscale, comme les machines virtuelles et les réseaux

Le state de Terraform

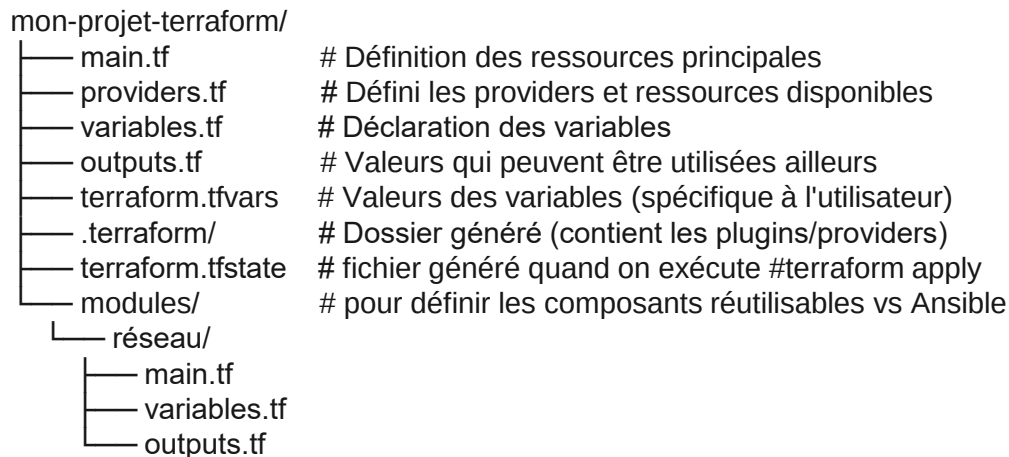


- L'état Terraform, ou state, est un fichier JSON qui enregistre l'état actuel de l'infrastructure gérée par Terraform.
- Par défaut, l'état Terraform est stocké dans un fichier nommé **terraform.tfstate**
- Il contient des informations cruciales telles que :
 - Les ID des ressources: Chaque ressource créée par Terraform possède un ID unique qui est enregistré dans l'état.
 - Les dépendances entre les ressources: Terraform utilise l'état pour comprendre les relations entre les différentes ressources et ainsi gérer les mises à jour de manière cohérente.
 - La configuration actuelle: L'état contient une version de la configuration Terraform au moment de la dernière application.

Les fichiers de configuration

- Ce sont du code écrit en langage de configuration HashiCorp (HCL)
- HCL utilise une syntaxe basée sur des clés-valeurs
- Pour un projet très simple, on peut utiliser seulement le fichier main.tf, sinon on peut adopter une structure qui favorise la réutilisation.
- le fichier main.tf

- ✓ C'est le fichier principal où sont définies les configurations de base de l'infrastructure à provisionner
- ✓ il contient les ressources, les modules, et les providers nécessaires pour déployer l'infrastructure.
- ✓ Il permet la configuration des dépendances entre les ressources.



Les fichiers de configuration

- Exemple de variables.tf:

- variables.tf

```
variable "ami_id" { description = "ID de l'image AMI  
pour l'instance EC2" # type = string # default = "ami-  
0c02fb55956c7d316" # }
```

```
# Déclaration de la variable pour l'image système (AMI)  
variable "ami_id" {  
  description = "ID de l'image AMI (amazon linux 2) pour l'instance EC2"  
  type = string  
  default = "ami-0c02fb55956c7d316" } #correspond à Amazon Linux 2 dans la région us-east-1
```

- main.tf

```
resource "aws_instance" "calculatrice_server" {  
  ami = var.ami_id  
# instance gratuite, 1 CPU, 1 GO RAM, performance réseau basse à modérée  
  instance_type = "t2.micro"  
  tags = {  
    Name = "Serveur-Calculatrice"  
  }  
}
```

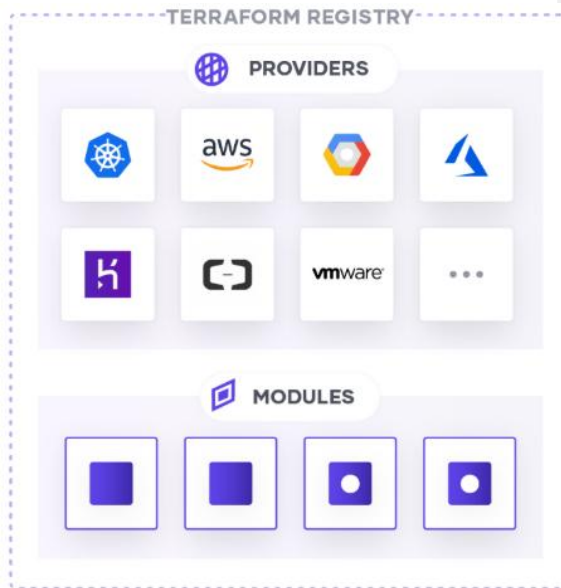
Data source

- les Data Sources de Terraform et le mécanisme de Gather Facts dans Ansible, mais avec une différence de fonctionnement importante.
 - Récupère des informations sur des ressources existantes dans le cloud (Ansible récupère les propriétés de la machine cible)
 - Facilite la création de nouvelles ressources en fonction de celles déjà existantes (ex. : récupérer un ID de sous-réseau avant de créer une instance EC2).



Terraform Registry / Modules

- Comme dans ansible, les modules sont définis dans Terraform pour organiser et réutiliser des configurations.
- Ces **modules** sont des ensembles de configurations Terraform pré-écrits, conçus pour automatiser la création et la gestion de ressources cloud spécifiques.
- La **Terraform Registry** est une plateforme en ligne qui centralise et met à disposition une vaste collection de modules Terraform créés par la communauté ou par des entreprises.
- Si on veut déployer un **cluster Kubernetes** sur AWS, on recherche un module Kubernetes dans la Terraform Registry et on l'utilise directement dans la configuration au lieu d'écrire tout le code manuellement.



Exemple de fichier de configuration

```
# Spécifie le Fournisseur AWS,
# la région où les ressources AWS seront créées
provider "aws" {
  region = "us-west-2"
}
#définit une data source pour récupérer
#des informations sur un sous-réseau public
data "aws_subnet" "public" {
  vpc_id = "vpc-12345678"
  cidr_blocks = ["10.0.1.0/24"]
}
# Ressource : Instance EC2
resource "aws_instance" "web_server" {
  ami          = "ami-08d4ac5b634553e16" #ubuntu 22. 04 LTS
  instance_type = "t2.micro"
  subnet_id    = data.aws_subnet.public.id
}
#l'adresse publique de l'instance EC2 sera
#attribuée automatiquement par AWS lors du déploiement
output "instance_ip" {
  value = aws_instance.web_server.public_ip
}
```

main.tf

Commandes CLI :

```
terraform init
Terraform plan
terraform apply
```

Type d'instance EC2 t2.micro
vCPU : 1 unité de CPU virtuel.
RAM : 1 Go de RAM.
Stockage : Ne comprend pas de stockage par défaut, mais peut utiliser des volumes EBS (Elastic Block Store).



Avantages de Terraform



Automatisation

- Permet d'automatiser le provisionnement et la gestion de l'infrastructure:
 - ✓ Moins d'erreurs humaines
 - ✓ Rapide déploiement.



contrôle des changements

- Facilite le suivi des modifications apportées à l'infrastructure au fil du temps, permettant ainsi un contrôle de version.



Scalabilité

- Permet de gérer l'infrastructure à grande échelle
 - ✓ ajout ou réduction rapide des ressources
 - ✓ Modification par le code



Flexibilité et portabilité

- Terraform est agnostique au fournisseur grâce à son système de providers.
- le même code Terraform pour AWS, Azure, GCP.

Synthèse des outils DevOps

Outil	Définition	Objectif principal	Cas d'utilisation	Avantages	Inconvénients
Docker	Outil de conteneurisation permettant d'exécuter des applications dans des environnements isolés (conteneurs).	Fournir des conteneurs légers et portables.	Déploiement d'applications isolées, tests dans des environnements uniformes, microservices.	- Léger et rapide - Portabilité multiplateforme - Large écosystème d'images sur Docker Hub - Idéal pour les microservices.	- Nécessite une orchestration pour les environnements complexes - Moins adapté pour les systèmes état-centrés (stateful).
Kubernetes	Système d'orchestration de conteneurs open source qui automatise le déploiement, la mise à l'échelle et la gestion des conteneurs.	Orchestrer des conteneurs à grande échelle.	Gestion d'applications distribuées, scalabilité automatique, haute disponibilité.	- Scalabilité automatique - Gestion simplifiée des déploiements complexes - Écosystème riche (Ingress, volumes, secrets, etc.).	- Complexité initiale élevée - Peut être surdimensionné pour les petites applications.
Jenkins/GitLab	Serveur d'intégration et de livraison continues (CI/CD) open source.	Automatiser les pipelines CI/CD.	Intégration continue, déploiement continu, tests automatisés, builds d'applications.	- Hautement extensible avec des plugins - Compatible avec de nombreux outils et technologies - Open source et gratuit.	- Gestion des plugins peut devenir complexe - Interface utilisateur moins intuitive comparée à des solutions modernes comme GitHub Actions ou GitLab CI.
Ansible	Outil d'automatisation de la gestion de configuration et d'orchestration basé sur YAML.	Automatiser les configurations et les déploiements.	Configuration des serveurs, déploiement d'applications, provisionnement d'infrastructures.	- Simple à apprendre grâce à YAML - Agentless (pas besoin d'installer un agent sur les cibles) - Large communauté et extensibilité.	- Moins performant pour les tâches très complexes ou nécessitant une orchestration avancée - Pas idéal pour la gestion d'infrastructure "as code".
Terraform	Outil d'infrastructure en tant que code (IaC) permettant de gérer des infrastructures de manière déclarative.	Provisionner et gérer des infrastructures.	Création et gestion des ressources cloud (AWS, Azure, GCP), infrastructures hybrides, gestion de réseaux complexes.	- Gestion déclarative des ressources - Compatible avec plusieurs fournisseurs (multi-cloud) - Gestion d'état avec des fichiers .tfstate.	- Courbe d'apprentissage plus élevée pour les débutants - Gestion d'état centralisée nécessite des configurations supplémentaires pour les équipes.

Synthèse des outils Cloud

Outil	Définition	Objectif principal	Cas d'utilisation	Avantages	Inconvénients
ESXi	Hyperviseur bare-metal développé par VMware qui permet de virtualiser des serveurs physiques.	Virtualisation d'infrastructure physique.	.Héberger plusieurs machines virtuelles (VM) sur un seul serveur physique.	- Performant et stable. - Administration via vSphere.- Support pour de nombreux systèmes d'exploitation	- Payant pour les fonctionnalités avancées. - Courbe d'apprentissage initiale.
vSphere Client	Interface graphique pour gérer les hôtes ESXi et leurs machines virtuelles.	Administrer et superviser les environnements ESXi.	Gestion des VM, surveillance des ressources, migration de VM (vMotion).	- Interface conviviale. - Accès centralisé à la configuration et à la supervision des VM.	- Dépendance à VMware pour la plateforme.
OpenDaylight	Plateforme SDN open source basée sur OpenFlow qui permet de gérer les réseaux programmables.	Automatiser et contrôler les réseaux via SDN.	Contrôleurs SDN dans les centres de données, gestion réseau automatisée, optimisation de trafic.	- Plateforme modulaire et extensible. - Supporte OpenFlow et d'autres protocoles.	- Configuration complexe. - Nécessite des compétences avancées en SDN.
Mininet	Simulateur de réseau SDN (Software Defined Networking) permettant de créer et tester des topologies réseau.	Simuler des environnements réseau pour la recherche et le développement.	Tester des architectures SDN, vérifier des protocoles réseau, former des étudiants.	- Léger et facile à utiliser. - Compatible avec OpenFlow.	- Limité aux tests. - Moins performant pour des réseaux complexes ou réels.
RAID	Technologie combinant plusieurs disques physiques en une seule unité logique pour redondance ou performance.	Améliorer la redondance ou les performances des systèmes de stockage.	Serveurs, NAS, systèmes critiques nécessitant une haute disponibilité ou des performances accrues.	- Sécurité accrue (RAID 1, 5, 6). - Performances augmentées (RAID 0, 10).	- Coût supplémentaire en matériel. - Certains niveaux (ex. RAID 0) n'offrent pas de redondance.
RDS (Bureau à Distance)	Technologie Microsoft permettant l'accès à distance aux bureaux Windows via le protocole RDP.	Fournir un accès distant aux bureaux et applications Windows.	Accéder aux postes de travail ou serveurs depuis n'importe quel appareil connecté.	- Accès distant simplifié. - Multi-utilisateur sur serveur.	- Nécessite une bonne configuration réseau. - Peut être lent sur des connexions à faible débit.



Fin du module